

Rapport de Projet - Développement d'un Moteur de Jeu 2D avec LibGDX

MarcheOuCrèveEngine2

Équipe et Contributions

- **Rathgeber-Kivits Eloi** [« Développeur de la state machine », « Développeur de l'IA »]
- **Darnaudguilhem Mathéo** [« Développeur des états du jeu », « Gestion de l'architecture du projet et de l'UML »]
- **Durand Gaïa (Florian)** [« Développeuse de l'UI », « Développeuse de la gestion des données Tiled »]

Section 1. Introduction

Ce projet est un moteur de jeu pour Tactical RPG 2D, un style de jeu dans lequel on contrôle plusieurs personnages au tour par tour, en les déplaçant sur une grille pour affronter une foule d'ennemis.

Le but est de permettre à des personnes de créer leur jeu sans avoir besoin de coder, en utilisant le logiciel Tiled comme interface. Ainsi, le but a été de faire le moteur avec le plus de fonctionnalités possibles pour permettre une grande liberté de création. Cela passe par la création de maps personnalisée, mais aussi la gestion des animations, des attaques etc.

Section 2. Présentation du projet

Présentez votre projet ici, les éléments suivants devront apparaître dans votre rapport. Vous pouvez évidemment adapter cette section par rapport à votre projet en ajoutant autant de sous-sections dont vous avez besoin.

Technologies et Outils Utilisés

- **LibGDX 1.14, Java 17** : pour le développement du moteur de jeu.
- **Tiled** : pour la création et la configuration des cartes.
- **GitHub** : pour témoigner de nos avancées et faire une road map.
- **Code-server** : pour le travail collaboratif (souvent en temps réel avec Live Share), en s'occupant de la compilation sur le serveur puis en ayant un script pour le récupérer depuis un Nginx et l'exécuter en local pour tester à chaque fois. Ça nous a aussi permis de faire des « save-states » à différents points clés. Tous les commits Git étaient du coup faits depuis ce VPS avec le même nom d'utilisateur, mais toutes les modifications étaient évidemment issues du fruit du travail du groupe entier.
- **PlantUML** : pour obtenir des diagrammes UML depuis du langage descriptif, ce qui nous a permis de réaliser bien plus facilement des diagrammes complexes.

Fonctionnalités implémentées.

Nos principales fonctionnalités consistent en l'usage avancé que nous avons fait de Tiled. Ainsi, il est possible de personnaliser les animations des personnages, ses statistiques et ses capacités, ainsi que créer des nouvelles attaques dont on peut modifier les propriétés et la zone d'effet.

Configuration et Ajout de Contenu avec Tiled

Tout le processus est expliqué en détail dans le README, accessible sur notre [GitHub](#).

Compilation et exécution

Tout est expliqué en détails dans le README, accessible sur notre [GitHub](#).

Les assets de jeu utilisés dans la démo sont tous libres de droits.

Section 3. Présentation technique du projet et contributions

Architecture Générale du moteur de jeu

Le jeu est modélisé par la classe MCGame, qui est donc la classe centrale qui fait tourner le jeu (MC est l'acronyme pour montrer que cette classe a été créée spécialement pour le moteur MarcheOuCrèveEngine2). L'état de celui-ci (si l'on est en mode Exploration ou Combat par exemple) est gérée grâce aux classe héritées de MCGameState. Le jeu possède également un EventBus, qui sert à centraliser les événements, un InputHandler, qui traite les inputs du joueur, un CameraManager pour pouvoir gérer différents mode de caméra etc. Tous les éléments du jeu héritent de la classe MCEntity, qui symbolise donc un objet dans le jeu, avec une position, et sont répertoriés dans le MCEntityManager. Aussi, le moteur possède de nombreuses classes afin de gérer l'UI du jeu.

Le jeu implémente donc le design pattern MVC, car les données (Model) sont stockées dans des classes distinctes de la logique du jeu (Controller) qui est elle-même distincte de l'affichage (View), géré par des classes spécifiques et les fonctions *render()* (*toutes distinctes des update(delta), les render() n'ayant même jamais accès au delta-time du jeu*).

Pour information, les Warnings au lancement de MCE2 sont du avec notre gestion de la librairie LibGDX.

Diagrammes UML

Les diagrammes UML se trouvent dans le dossier « uml » à la racine du projet.

Utiliser et étendre la librairie du moteur de jeu

Notre librairie est composée de nombreux packages :

- **SM** : Ce package contient les différents states créés pour le jeu ainsi que pour la gestion des entités. Il contient également les states machines nécessaires à leur utilisation. **Design patterns : State**
- **Entities** : Ce package contient toutes les entités du jeu. La principale utilisation de cette classe est sa classe dérivée MCCharacter, qui symbolise un personnage, allié comme ennemi, et gère donc toutes ses options, contient sa propre state machine etc. Les portails permettant de changer de map et les projectiles héritent également de MCEntity. Enfin, ce package comporte les attaques utilisées par les MCCharacter. **Design pattern : Factory**

- **Input** : Ce package contient principalement la classe MCInputManager, chargée de recevoir, traiter et diffuser grâce à l'EventBus les inputs reçus. **Design patterns** : **Command, Singleton**
- **Shared** : Ce package contient les classes nécessaires à tout le projet, comme des fonctions utiles, ou des types personnalisés (comme MCIntVector2). Mais il contient surtout le MCEventBus, chargé de gérer les événements à travers le jeu. **Design patterns** : **Observer, Singleton, Flyweight**
- **Tiledmap** : Ce package contient les classes pour stocker et lire les maps venant de Tiled. Il contient aussi le singleton MCPathFinder, qui implémente les algorithmes de Bresenham et A*, afin de trouver les chemins/trajectoires adéquates. **Design patterns** : **Singleton**
- **Cameras** : Ce package contient les différents modes de caméras, ainsi que le singleton MCCameraManager qui permet de gérer et de changer de mode. **Design patterns** : **Strategy, Singleton**
- **UI** : Ce package contient toutes les classes nécessaires pour gérer et afficher l'un du jeu.
- **Screens** : Ce package contient MCGameScreen, notre propre implémentation de la classe Screen de LibGDX.
- **Exceptions** : Ce package contient toutes les erreurs personnelles que nous avons utilisées. Certaines sont plus utiles que d'autres.
- **AI** : Ce package contient la classe MCAI, qui sert aux ennemis à prendre les décisions. Celle-ci se base sur un système de points donnés aux actions, en fonction des dégâts qu'elles permettent de faire et les risques qu'elles font prendre. **Design pattern** : **Strategy (avec MCEntity)**
- **Capacities** : Ce package contient la classe MCEffect. Cela représente un effet applicable à un personnage, qui modifie son comportement ou ses statistiques. Par exemple, se déplacer plus vite ou avoir un bouclier sont des effets. Ceux-ci sont applicables grâce à des MCCapacities.
- **Vehicles** : Ce package ne contient qu'une seule interface, mais elle est sans nul doute essentielle.

Pour étendre notre moteur, il est important de se référer aux classes de base. Ainsi, il peut être intéressant d'ajouter de nouvelles actions aux personnages, cela se faisant en créant de nouveaux MCCharacterState. Les packages contiennent normalement tout ce qu'il faut pour étendre le code proprement, et de manière flexible, grâce aux singletons et à l'EventBus.

Répartition des Tâches

Disclaimer : *Ce projet a été réalisé intégralement en groupe. Même si certains se sont spécialisé sur différents aspects, nous sommes tous intervenus dans les parties des autres, et avons un droit de regard sur tout ce qui était produit. Ainsi, nos réussites sont le fruit d'une réflexion collective, et nous sommes tous responsables des erreurs commises. Si cela est possible, nous aimerions que cela soit pris en compte et que nous ayons la même note finale.*

Rathgeber Eloi :

Je me suis principalement chargé de la mise en place des states machines, puis de la création des MCCharacterState. J'ai également travaillé sur le système d'IA du jeu, le système d'effets ou encore le singleton MCPathfinder. Enfin, j'ai participé à l'élaboration du MCEventBus. J'ai aussi écrit la majorité de la Javadoc (d'ailleurs en anglais, pour respecter les bonnes pratiques).

Durand Gaïa :

Je me suis occupée de la gestion des caméras (MCameraManager, pattern Strategy), des entrées clavier/souris (MCInputManager, pattern Command), de la conception de l'interface utilisateur (HUD, menus) qui m'a amené à devoir gérer les ressources partagées (MCSharedAssets). J'ai aussi implémenté la gestion des projectiles.

Darnaudguilhem Mathéo :

J'ai conçu la partie « Exploration » du jeu (ExplorationPlayer, états d'exploration), ainsi que l'intégration des portails (MCPortal) avec sa gestion des passages entre les différentes maps (selon leurs contraintes et leur mode de Map, des gestion qui se trouvent principalement dans MCGame et MCEntityManager). J'ai également géré la conception et l'organisation des maps et des tilesets dans Tiled.

Section 4. Conclusion et Perspectives

Nous avons atteint un niveau satisfaisant pour le projet. En l'état, notre moteur permet de créer un Tactical-RPG simple, et une expérience déjà bien modulable. La prise en charge de Tiled, ainsi que la programmation du core gameplay ont été des défis que nous sommes fiers d'avoir réussi.

Toutefois, il reste plusieurs pistes d'améliorations. La première est d'étoffer les possibilités, donner plusieurs IA possibles aux ennemis, plus de capacités ou d'attaques pour les personnages, ajouter des obstacles destructibles, des événements spéciaux etc. La deuxième est d'ajouter la dimension RPG, avec de l'expérience, des possibilités d'évolutions, des menus pour gérer cet aspect.